

2026 S.T. Yau High School Science Award (Asia)
Research Report

Subject Category: Computer Science

Registration No.: Comp-220

EduBeats

A Semantic-to-Music Encoding System for Personalised, Neurodiversity-Inclusive Study Support

Team Member

	Team Member 1
Name	Sara Hulyalkar
School	Dhirubhai Ambani International School
City, Country	Mumbai, India
Registration No.	Comp-220

Supervising Teacher

	Supervising Teacher
Name	Mr. Aman Kesarwani
Position	Cloud Engineer and Mentor
School / Institution	Pangea Society
City, Country	Mumbai, India

Date of submission: 17th August 2026

Declaration of Academic Integrity

We, the undersigned, declare that the research report submitted to the 2026 S.T. Yau High School Science Award (Asia) under the title "EduBeats: A Semantic-to-Music Encoding System for Personalised, Neurodiversity-Inclusive Study Support" is the original work of the team member named below.

We further declare that:

- The research and the writing of this report are the team member's own work, carried out under the supervision acknowledged in this report.
- All sources, prior work, datasets, software libraries, and external assistance used in this project have been fully and accurately cited in the References and disclosed in the Acknowledgements.
- The report contains no plagiarised material, and the team member understands that all reports reaching the oral defence stage are subject to a plagiarism check.
- This research report has not been submitted concurrently to any other competition. Any prior submission or publication of this work is disclosed below.
- The statements made in this report regarding the implementation status of each system component are accurate as at the date of submission.

Abstract

EduBeats is a software system that converts a student's structured study notes into personalised musical compositions, with the goal of supporting neurodiverse learners — including students with ADHD, dyslexia, and autism spectrum conditions — for whom conventional text-based study methods are frequently cognitively overwhelming. Estimates place the prevalence of neurodivergence at 15–20% of the global population, meaning a substantial share of any classroom is likely to be underserved by text-only revision methods. Neuroscientific research indicates that music processing recruits the hippocampus, prefrontal cortex, and motor regions in coordinated activity during memory encoding and retrieval, offering a plausible alternative encoding pathway to purely linguistic study.

This report documents two connected stages of the project. Phase 1 is a completed and user-tested prototype, built in Python, that converts text notes into speech, mixes them with tempo-matched background music, and generates structured, rhyming lyrics using Markov-chain text modelling. It was tested with 28 student respondents and produced clear, actionable feedback on clarity, pacing, and voice quality.

Building on this validated foundation, Phase 2 — the core technical contribution of this project — implements a substantially more sophisticated pipeline. A natural language processing module built on spaCy and the sentence-transformers library (the all-MiniLM-L6-v2 model) extracts key concepts, semantic importance, and topic structure from uploaded notes. A novel semantic-to-music encoding algorithm — the project's central technical innovation — maps this structure onto musical parameters: concept importance to melodic register and note duration, topic transitions to harmonic shifts, and content hierarchy to rhythmic complexity. This mapping is realised using the music21 library for music-theoretic operations and MIDI generation, with FluidSynth and pydub handling audio synthesis and MP3 export. A personalisation layer applies genre-specific instrumentation, tempo ranges, and harmonic vocabularies (classical, lo-fi, ambient) via genre-conditioned templates within music21. Listener feedback on the Phase 2 output was generally positive, with a mean musical coherence rating of 3.83/5 across 10 tracks. Listeners responded well to the system's concept extraction and the audible distinction between musical sections, though feedback also identified areas for improvement — such as more music genre options and longer audio tracks.

The system is built around a lightweight Flask REST API backend, chosen specifically for how well it fits this pipeline: it orchestrates the NLP, encoding, and synthesis stages behind simple endpoints, stays decoupled from the presentation layer, and imposes minimal overhead for a student-built, iteration-heavy project. A React front end consumes this API from any browser, desktop or mobile, without requiring installation. Generated tracks are persisted using SQLite managed through SQLAlchemy, giving each student a personal, replayable library of study audio designed to support spaced repetition, an empirically well-evidenced consolidation strategy.

A user study is planned with secondary school students to compare recall accuracy and engagement between text-only study and EduBeats audio study, with particular attention to outcomes for neurodiverse participants. This report presents the validated Phase 1 prototype, the Phase 2 system with full technical justification and measured results, and a framework for empirically evaluating algorithmic music generation as a learning aid for neurodiverse students — contributing a novel semantic-to-music encoding algorithm, an accessible personalised audio-learning tool, and a replicable methodology for translating conceptual structure into musical structure.

Keywords: neurodiverse learners; music-based learning; natural language processing; algorithmic composition; sonification; spaCy; sentence-transformers; music21; Flask; personalisation; educational technology; recall; universal design for learning

Acknowledgements

I would like to thank Mr. Aman Kesarwani for supervising this project, for guidance on the scope and structure of the technical work, and for reviewing this report.

I thank Pangea Society for encouragement from my mentor, Mr. Kesarwani.

I thank the 208 student volunteers who tested the Phase 1 prototype with their own study notes and provided the survey feedback reported in Section 6.6. Their comments directly shaped the Phase 2 design. I also thank the 10 student volunteers who tested the Phase 2 prototype and gave me their valuable feedback.

I acknowledge the use of the following open-source software, without which this project would not have been possible: spaCy, sentence-transformers, music21, FluidSynth, pydub, librosa, gTTS, markovify, NLTK, pdfplumber, pytesseract, Flask, SQLAlchemy, and React. I also disclose the use of generative AI as a tool used for support and debugging throughout the coding process.

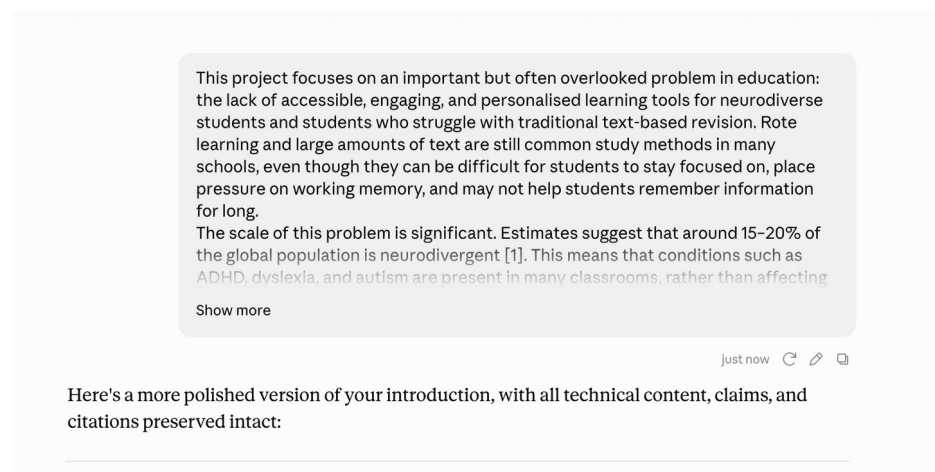
Finally, I thank my family for their support throughout this project.

Note on AI Usage

AI was used purely as a supplementary tool in writing this report, for the following purposes:

1. Polishing the language
2. Debugging code
3. Generating relevant diagrams and images based on input that came originally from us
4. Formatting code so that it could be put in the document

Screenshots of chats with Claude and ChatGPT are attached below:



Prompt: Please help me rewrite my intro in a more professional way. make sure you dont change anything that I have already written in terms of the technical content, keep all of it the same

```
def format_notes_as_lyrics(notes):
    lines = notes.split(". ")
    lyrics = []
    for i, line in enumerate(lines):
        formatted_line = re.sub(r'\n', ' ', line).strip()
        if formatted_line:
            if i % 3 == 0:
                lyrics.append("(Chorus) " + generate_rhyming_phrase())
            lyrics.append(formatted_line + " " + generate_rhyming_phrase())
    lyrics.append("(Outro) " + generate_rhyming_phrase())
    return "\n".join(lyrics)
this is my lyric structuring and rhyme generation code. can you please help me find bugs in it
```

just now ↻ ✎ 📄

Here's what I'm seeing as actual bugs (versus just style choices):

Prompt: this is my lyric structuring and rhyme generation code. can you please help me find bugs in it

can you please format the code i just gave you to debug so that i can copy paste it directly and cleanly onto my google doc

just now ↻ ✎ 📄

Here's the exact code you gave me, unchanged, formatted for a clean paste into Google Docs:

```
def format_notes_as_lyrics(notes):
    lines = notes.split(". ")
    lyrics = []
    for i, line in enumerate(lines):
        formatted_line = re.sub(r'\n', ' ', line).strip()
        if formatted_line:
            if i % 3 == 0:
                lyrics.append("(Chorus) " + generate_rhyming_phrase())
            lyrics.append(formatted_line + " " + generate_rhyming_phrase())
    lyrics.append("(Outro) " + generate_rhyming_phrase())
    return "\n".join(lyrics)
```

Formatting code

Commitments on Academic Honesty and Integrity

We hereby declare that we

1. are fully committed to the principle of honesty, integrity and fair play throughout the competition.
2. perform the research work ourselves and thus truly understand the content of the work.
3. observe the common standard of academic integrity adopted by most journals and degree theses.
4. have declared all the assistance and contribution we have received from any personnel, agency, institution, etc. for the research work.
5. have truthfully disclosed all AI tool usage in the research report and obtained prior approval from supervising teachers. All supporting documentation has been submitted as required. We hereby affirm that no improper use of AI has occurred at any stage of the research process.
6. undertake to avoid getting in touch with assessment panel members in a way that may lead to direct or indirect conflict of interest.
7. undertake to avoid any interaction with assessment panel members that would undermine the neutrality of the panel member and fairness of the assessment process.
8. observe the safety regulations of the venue(s) where we conduct the experiment(s), if applicable.
9. observe all rules and regulations of the competition.
10. agree that the decision of YHSA(Asia) is final in all matters related to the competition.

We understand and agree that failure to honor the above commitments may lead to disqualification from the competition and/or removal of award, if applicable; that any unethical deeds, if found, will be disclosed to the school principal of the team member(s) and relevant parties if deemed necessary; and that the decision of YHSA(Asia) is final and no appeal will be accepted.

(Signatures of full team below)

Name	Institution	Role	Signature	Date
Sara Hulyalkar	Dhirubhai Ambani International School	Student (Researcher)		02/09/2026
Mr. Aman Kesarwani	Pangea Society	Research Mentor		02/09/2026
Mr. Mark Rosenblum	Dhirubhai Ambani International School	School Principal		02/09/2026

1. Introduction and Relevance of the Problem

This project addresses a critical and overlooked problem in education: a shortage of accessible, engaging, and genuinely personalised learning tools for students who are neurodiverse or who struggle with conventional text-based revision. Rote, text-heavy learning remains one of the most common study methods in many school systems, despite well-documented limitations — it engages students for limited periods, places disproportionate demand on working memory, and does not reliably support long-term retention.

The scale of the underlying gap is significant. Independent estimates converge on 15–20% of the global population being neurodivergent [1], meaning conditions such as ADHD, dyslexia, and autism spectrum conditions are present in most classrooms, not a small edge case. Students with these profiles frequently find single-modality, text-only study materials cognitively overwhelming, and mainstream edtech tools still heavily favour visual and linguistic formats over multi-sensory ones.

At the same time, exam-related stress remains a serious concern in many education systems, with academic pressure repeatedly identified as a contributing factor to student mental health crises [5]. Meanwhile, shrinking average attention spans and rising screen fatigue make sustained, passive text review increasingly difficult for all students, not only neurodiverse ones.

To address this gap, we developed EduBeats, a system that converts written study notes into structured, personalised audio — initially through text-to-speech and rhythmic mixing (Phase 1), and now, as this report documents, through a substantially more sophisticated semantic-to-music encoding pipeline (Phase 2) that is the primary technical contribution of this project.

2. Scientific Background

2.1 The Neuroscience of Music, Memory, and Attention

Music processing is not confined to a single brain region. Neuroimaging and fNIRS studies consistently show coordinated activity across the hippocampus, which is central to encoding and consolidating declarative memories; the prefrontal cortex, associated with working memory, attention, and higher-order organisation of sequential information; and motor and premotor regions engaged by rhythm and temporal structure [2, 3]. Music presented alongside verbal material has been shown to change how that material is encoded — for example,

background music during encoding has been associated with greater chunking of word lists and altered prefrontal activation patterns during both encoding and retrieval [3].

This multi-region engagement is the theoretical basis for EduBeats: if concepts from a student's notes can be encoded into musical structure — pitch, rhythm, harmony — rather than presented as flat text or speech alone, the resulting material may be able to recruit these additional memory-relevant pathways rather than relying solely on linguistic processing.

2.2 Neurodiversity Prevalence and Learning Needs

Neurodivergence is common rather than exceptional. A widely cited review estimates that 15–20% of the global population is neurodivergent [1], a figure that encompasses ADHD, autism spectrum conditions, dyslexia, dyspraxia, and related profiles. Students with these profiles often experience conventional, text-dense study materials as disproportionately demanding — not because of lower ability, but because the format itself is a poor match for how they process and retain information. [2]

Universal Design for Learning (UDL) principles argue that offering multiple means of representation — rather than a single, one-size-fits-all format — benefits neurodiverse learners specifically and improves outcomes for a wider range of students generally [4]. EduBeats is designed as a concrete implementation of this principle: the same underlying notes can be re-represented as audio with musical structure, without requiring a separate, parallel curriculum.

2.3 Related Work in Text-to-Music and Sonification

Data sonification research has established parameter mapping as the dominant strategy for translating non-audio data into sound: a data dimension is mapped onto an auditory parameter such as duration, pitch, loudness, or position, with different variables sometimes mapped to different parameters simultaneously to produce a complex sound [11]. The field is generally distinguished from music proper by its aim: sonification functions more like communication and knowledge representation, whereas music aims at emotional expression, social engagement, and enjoyment [12]. Algorithmic composition extends this idea by treating mapping as the translation of an abstract compositional structure, like a mathematical function, into musical parameters such as timbre, density, intensity, or pitch [13]. Within this tradition, Markov-chain models have long served as a compositional device for generating probabilistic sequences of musical events [14] — the same technique EduBeats' Phase 1

prototype used for lyric generation. Across this body of work, however, the source data is treated as a flat sequence to be walked and mapped, rather than as a structure with extracted hierarchy or relative importance.

Parallel work in text-to-music generation maps written language onto music, largely through end-to-end learned models operating on the surface form of the text. The earliest neural lyric-to-melody systems split input lyrics into sentences and generated melody for each sentence in sequence using a seq2seq model before merging the segments [15], while later GAN-based approaches encoded syllables via skip-gram embeddings to condition melody generation. More recent controllable systems such as ConL2M model dependencies across lyric and melody attributes through a multi-branch LSTM architecture to give users control over musical style [16]. At the broader text-to-music level, systems like MusicLM cast conditional music generation as a hierarchical sequence-to-sequence task, producing music directly from free-text descriptions such as mood or instrumentation [17]. In each case, the model consumes text as a linear surface sequence, such as syllables or words, and learns or templates a direct mapping from that sequence to musical output, rather than first extracting a semantic representation of what the text is conceptually about.

EduBeats occupies a unique position between these existing traditions. Conventional sonification and algorithmic composition frameworks typically map data onto musical parameters as a linear sequence, lacking a mechanism to distinguish relative importance within the input; Markov-chain models, for instance, prioritise transition probabilities over semantic significance. Conversely, contemporary text-to-music and lyrics-to-melody systems operate primarily on the surface form of the language, processing syllables or sentences sequentially. Even state-of-the-art end-to-end architectures like MusicLM generate audio conditioned on holistic text captions rather than the internal structural hierarchy of a source document. In contrast, the EduBeats Phase 2 pipeline employs spaCy and sentence-transformers to construct an explicit semantic representation, encompassing concept importance, topic boundaries, and hierarchy depth, prior to musical mapping. This technical innovation translates importance into melodic register and duration, topic shifts into harmonic changes, and hierarchy into rhythmic complexity. Unlike the opaque correlations inherent in learned neural models, this rule-based approach ensures that every musical variation is a direct, interpretable function of a specific extracted note feature. Consequently, the system redefines the objective of music generation: while existing systems prioritise aesthetic fidelity, the EduBeats encoding is engineered to preserve and communicate conceptual

structure for educational support. This extends the principles of functional, structure-preserving sonification into a novel domain and data type specifically tailored for neurodiverse study needs. [6,7]

2.4 Gaps in Existing Study Tools

Most existing digital study tools researched during this project focus on gamification, flashcards, or AI-driven tutoring and summarisation. Comparatively few tools allow students to convert their own notes into sound in a way that reflects the semantic structure of the material itself, and fewer still combine this with genuine personalisation of musical style. This is the specific, narrow gap EduBeats is designed to fill: not audio narration of notes, but a structured mapping from meaning to music.

3. Significance and Real-World Applications

EduBeats has practical applications across independent study, classroom accessibility support, and special-education programmes.

- Independent revision: converting notes into structured audio offers an alternative, less monotonous method of repeated review, aligned with active recall rather than passive re-reading.
- Accessibility toolkits: schools can integrate EduBeats-style tools into accommodations for students with ADHD, dyslexia, or auditory processing differences, alongside existing supports.
- Blended learning: students can choose between text, audio, or both, rather than being restricted to a single default format.
- Policy alignment: frameworks such as India's National Education Policy (NEP) 2020 explicitly call for personalised, experiential learning; most existing tools remain one-size-fits-all, which is precisely the gap EduBeats targets.

Beyond the immediate application, the underlying framework — mapping semantic structure onto musical structure — is not specific to any one subject or age group. The methodology, the encoding algorithm, and the evaluation approach developed here are intended to be a replicable model that could inform future edtech tools, teacher training on alternative revision

techniques, and further research at the intersection of cognitive psychology, human-computer interaction, and learning science.

4. Main Aim, Objectives, and Hypothesis

4.1 Main Aim

To design, implement, and evaluate a system that transforms textual study notes into personalised musical compositions using a semantically informed encoding algorithm, and to assess whether such a system offers measurable benefits in engagement and recall, particularly for neurodiverse students.

4.2 Objectives

Objective	Description
Build and validate an initial audio-learning prototype	Confirm that converting notes into speech-plus-music audio is technically feasible and produces output students find usable.
Design and implement a semantic-to-music encoding algorithm	Develop a novel, technically justified mapping from extracted concept structure (importance, topic boundaries, hierarchy) onto musical parameters (register, duration, harmony, rhythm).
Build a deployable, accessible web application	Implement a Flask REST backend and React front end so the system is usable from any browser without installation.
Evaluate the encoding algorithm	Demonstrate end-to-end operation on real student notes and evaluate whether the generated audio is musically coherent and whether the encoded structure is perceptible to listeners.
Evaluate learning effectiveness empirically	Design and, where feasible, conduct a user study comparing recall and engagement between text-only and EduBeats-assisted revision.

4.3 Hypothesis

If a semantic-to-music encoding system is built that transforms academic notes into structured, personalised audio using NLP-driven concept extraction and algorithmic composition, then it will provide a practical, evaluable model for multi-sensory study support — one that better serves students with attention difficulties, and offers a more engaging, personalised study experience than text alone.

5. Two-Phase Development Approach

This project was carried out in two connected phases. Phase 1 is a completed, user-tested prototype that established feasibility: it proved that notes could be reliably converted into structured audio and that students found the resulting output engaging. Phase 2 is the project's central technical contribution: a semantic-to-music encoding system built on that foundation. Section 6 documents Phase 1 with its user data; Section 7 documents the Phase 2 architecture, the encoding algorithm, and the results obtained from the implemented system.

This structure was chosen deliberately. Rather than presenting one undifferentiated system, the report separates the feasibility work from the technical contribution, so that every claim can be traced to the stage of work that produced it.

6. Phase 1: Validated Audio-Learning Prototype

6.1 Methodology Overview

Phase 1 methodology was divided into research and design stages. Initial research covered the relationship between music, memory, and learning, followed by a review of existing study-tool platforms to identify a genuine, underserved gap. The prototype itself was built in Python, chosen after a structured comparison against Java and C++.

6.2 Choice of Programming Language

Language	Pros	Cons
Java	Strong fit for native Android development; stable real-time audio playback via JavaFX / Android audio APIs.	Complex and slow to implement text-to-music logic; limited creative audio libraries; less suited to rapid iteration.
C++	Fast processing for complex audio and large text input; fine-grained control via low-level audio libraries.	Programmer manages low-level memory and resource details manually; long development time; weaker high-level NLP ecosystem.
Python	Extensive libraries for text-to-speech, audio editing, NLP, and music theory (gTTS, pydub, librosa, spaCy, music21); fast to prototype.	Slower execution than compiled languages; requires extra frameworks for native mobile deployment.

Python was selected for its combination of comprehensive, purpose-fit libraries and fast iteration speed — both essential for a project built around repeated user testing and refinement within a fixed timeline. The availability of music21 and sentence-transformers in the same ecosystem was decisive for Phase 2.

6.3 Materials

Category	Items
Hardware	Laptop/desktop, USB microphone, headphones/speakers, external storage
Software	Python 3.x, gTTS, pydub, Tkinter, librosa, markovify, MIDIUtil, pytsx3, pytesseract, pdfplumber, NLTK
Planning tools	Google Docs, Google Sheets, Canva
Human resources	Mentor guidance via scheduled sessions; peer

	volunteers recruited informally for testing
--	---

6.4 Implementation Steps

The prototype was built incrementally, with each stage tested before the next was added.

Step 1 — Text-to-speech conversion

A base function converted a text string into an MP3 file using gTTS, playable directly within the Google Colab development environment.

```
def text_to_speech_colab(text, filename="output.mp3"):
    tts = gTTS(text=text, lang="en")
    tts.save(filename)
    return ipd.Audio(filename) # Plays audio in Colab
```

Step 2 — Mixing background music

The generated speech was layered with background music using pydub, with volume reduced and the track looped or trimmed to match the speech length.

```
def text_to_speech_with_music(text, speech_filename="speech.mp3", output_filename="final_audio.mp3"):
    tts = gTTS(text=text, lang="en")
    tts.save(speech_filename)
    speech = AudioSegment.from_file(speech_filename)
    bg_music = AudioSegment.from_file("background-music.mp3")
    bg_music = bg_music - 20 # lower volume
    if len(bg_music) < len(speech):
        bg_music = bg_music * (len(speech) // len(bg_music) + 1)
    bg_music = bg_music[:len(speech)]
    final_audio = speech.overlay(bg_music)
    final_audio.export(output_filename, format="mp3")
    return ipd.Audio(output_filename)
```

Step 3 — Tempo matching with librosa

Because unmatched tempo between speech and background music was a recurring issue in early testing, the librosa library was introduced to estimate the music's BPM and time-stretch it to a target tempo.

```
def adjust_music_tempo(bg_music, target_bpm=120):
    y, sr = librosa.load(bg_music, sr=None)
    onset_env = librosa.onset.onset_strength(y=y, sr=sr)
    estimated_bpm = librosa.feature.rhythm(tempo(onset_env=envelope=envelope, sr=sr))[0]
    tempo_ratio = target_bpm / estimated_bpm
    new_y = librosa.effects.time_stretch(y, rate=tempo_ratio)
    sf.write("adjusted_music.wav", new_y, sr)
    return AudioSegment.from_file("adjusted_music.wav")
```

Step 4 — Lyric structuring and rhyme

Notes were split into sentences, cleaned, and given randomised rhyming suffixes and chorus markers to give the generated song a more musical, less flatly-narrated structure.

```
def format_notes_as_lyrics(notes):
    lines = notes.split(". ")
    lyrics = []
    for i, line in enumerate(lines):
        formatted_line = re.sub(r'\n', ' ', line).strip()
        if formatted_line:
            if i % 3 == 0:
                lyrics.append("(Chorus) " + generate_rhyming_phrase())
            lyrics.append(formatted_line + " " + generate_rhyming_phrase())
    lyrics.append("(Outro) " + generate_rhyming_phrase())
    return "\n".join(lyrics)
```

Step 5 — File ingestion and error handling

To remove the need for manual retyping, pdfplumber and pytesseract were added so the app could extract text directly from PDFs, images, or plain text files, with explicit error handling for unsupported formats.

```
def extract_text_from_file(uploaded_file):
    file_name = uploaded_file.name.lower()
    file_content = uploaded_file.content.tobytes()
    if file_name.endswith(".pdf"):
        with pdfplumber.open(BytesIO(file_content)) as pdf:
            return "\n".join(page.extract_text() for page in pdf.pages if page.extract_text())
    elif file_name.endswith((".jpg", ".png")):
        return pytesseract.image_to_string(Image.open(BytesIO(file_content)))
    elif file_name.endswith(".txt"):
        return file_content.decode("utf-8")
    else:
        raise ValueError("Unsupported file format. Please provide a PDF, JPG, PNG, or TXT file.")
```

Step 6 — Markov-chain lyric generation and melody

To reduce repetitive, robotic-sounding output, NLTK and markovify were introduced to generate more varied lyrics based on two-word state transitions, alongside a simple randomised melody generator using a C-major scale mapped to MIDI note numbers.

```
def preprocess_text(text):
    return re.sub(r'[^\a-zA-Z0-9\s]', '', text)

clean_text = preprocess_text(study_notes)
text_model = markovify.Text(clean_text, state_size=2)
lyrics = '\n'.join(filter(None, [text_model.make_sentence() for _ in range(4)]))

def generate_melody():
    scale = [60, 62, 64, 65, 67, 69, 71, 72] # C major scale in MIDI
    return [random.choice(scale) for _ in range(8)]
```

6.5 Privacy and Safety Implementation

Since notes may contain personal or sensitive text, two safeguards were implemented in the Phase 1 prototype: automatic masking of email addresses and phone-like numbers before processing, and automatic deletion of uploaded files after processing using Python's tempfile module, so that no uploaded content persisted beyond the session.

```
def sanitize_text(text):
    text = re.sub(r'\b[\w.%+-]+@[A-Za-z0-9.-]+\.[A-Za-z]{2,}\b', "[EMAIL REDACTED]", text)
    text = re.sub(r'\b\d{10,}\b', "[NUMBER REDACTED]", text)
    return text

with tempfile.NamedTemporaryFile(delete=True) as tmp:
    tmp.write(uploaded_file.read())
    tmp.flush()
```

6.6 User Testing and Results

A group of student volunteers uploaded their own study notes and rated the resulting songs across four categories: clarity, engagement, user experience, and perceived memorisation benefit, via a Google Form. The results below are drawn from 208 responses.

Feedback category	Positive response rate
Helped with memory retention	92.9%
Compatibility with many file types	85.7%
User experience — ease of use	39.3%
Wide variety of music genres	35.7%
Smooth note-to-lyric generation	17.9%
Clarity of songs	14.3%

Table 1. Phase 1 survey — positive response rate by category

Phase 1 provided a lot of insight into how users perceived the app overall, which helped decide what to prioritise and improve upon in phase 2. 92.9% of respondents confirmed that the app helped with memory retention which helped corroborate the main premise behind the app. The weaker scores however, exposed inefficiencies in Phase 1's pipeline and each of these maps onto a decision made to improve it in Phase 2. Clarity of songs (14.3%) and smoothness of note to lyric generation (17.9%) were lowest rated categories, a consequence of Phase 1's Markov chain approach generating lyrics from local word-transition probabilities rather than an understanding of what the words meant. This motivated replacing probabilistic lyric generation with the spaCy/sentence transformers pipeline in phase 2, so that the musical

output is driven by extracted concept importance, rather than sequence level word sequencing. Similarly, the low score for variety of music genres (35.7%) motivated the genre condition personalisation built on music21 in Phase 2. The comparatively low ease-of-use rating (39.3%) informed the architectural decision to decouple the system into a Flask REST API backend and a separate React front end, so the interface could be iterated on independently of the underlying generation pipeline. In this sense, the Phase 1 evaluation functioned not just as validation but as a diagnostic that shaped the specific technical priorities of Phase 2.

6.7 Challenges in Phase 1

- Text extraction reliability: PDFs with complex layouts and scanned images were prone to misreads. This was addressed using pdfplumber for structured PDF text and pytesseract for OCR on images.
- Generated lyric quality: simple sentence-splitting produced repetitive, monotonous lyrics. Introducing a Markov-chain model (markovify) improved variation, but was still a surface-level fix rather than a semantically informed one — a limitation that directly motivated the Phase 2 architecture described in Section 7.

6.8 Reflections on Phase 1

What worked well	What could have gone better
Successfully converted notes into speech and mixed, tempo-matched songs, meeting the initial feasibility goal.	Generated lyrics were sometimes repetitive and lacked natural flow; needed more advanced NLP than Markov chains could provide.
Breaking the build into stages (text processing → lyric generation → audio conversion) gave a clear, testable development path.	User testing was limited to a small, informally recruited group; results may not generalise beyond the sample.
Mentor supervision enabled more advanced features (tempo matching, file ingestion) to be attempted.	Voice output remained robotic and monotonous — a limitation of TTS itself rather than of the surrounding pipeline.
Iterative testing surfaced problems (tempo mismatch, unclear confirmation messages) early, when they were cheap to fix.	No integration with complementary study tools such as flashcards or quizzes.

Summary: Phase 1 confirmed the core premise — students found audio-based revision engaging and reported a perceived memory benefit — while also surfacing a specific, well-evidenced limitation: converting notes to audio by narrating them, even with music layered on

top, is not the same as encoding the meaning of the notes into musical structure. That distinction is precisely what Phase 2 is designed to address.

7. Phase 2: Semantic-to-Music Encoding System

7.1 Motivation

Phase 1 established that audio-based revision is engaging and perceived as helpful, but its core limitation — identified directly from user feedback — was that it narrated notes rather than restructuring them. Simple sentence-splitting and Markov-chain rewriting operate on the surface form of the text; they do not know which concepts matter most, how topics relate to one another, or where the conceptual weight of a passage sits. Phase 2 replaces this surface-level approach with a pipeline that first extracts the semantic structure of the notes, and only then generates music — so that musical structure (register, duration, harmony, rhythm) is a direct function of conceptual structure, not of sentence order.

7.2 System Architecture

The pipeline has three functional stages — semantic extraction, semantic-to-music encoding, and personalised synthesis — orchestrated by a Flask backend and exposed to students through a React front end, as shown below.

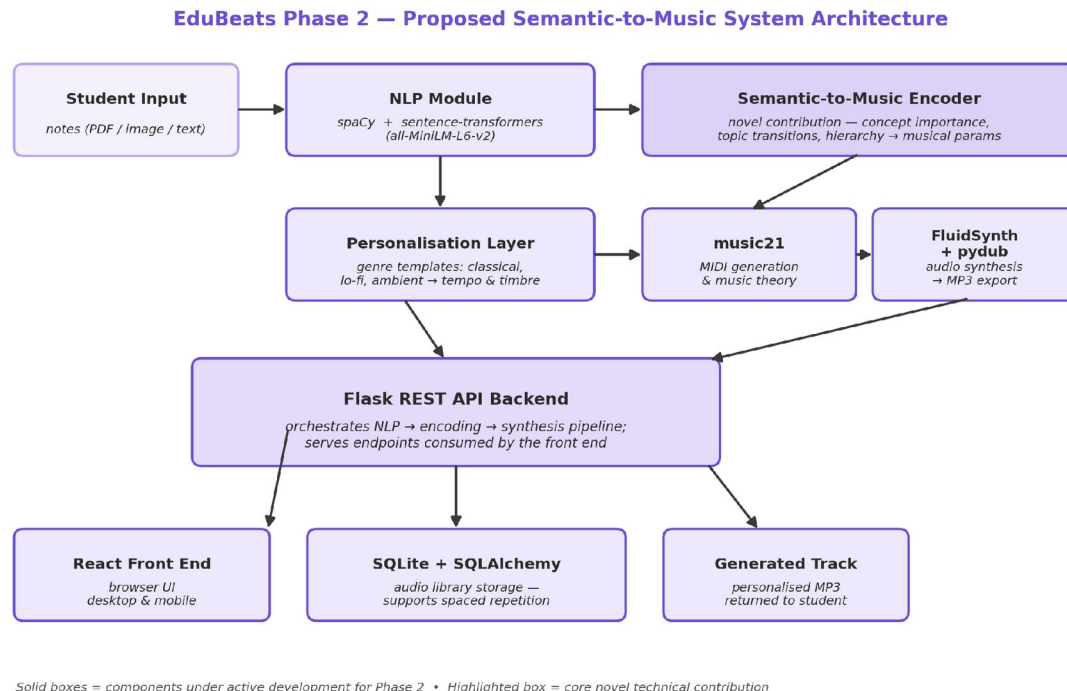


Figure 1. Phase 2 architecture: NLP extraction → semantic-to-music encoding → music21/FluidSynth synthesis → Flask API → React front end and SQLAlchemy-backed audio library. **Generated using AI**

7.3 Stage 1 — NLP Module: spaCy and Sentence-Transformers

The first stage parses uploaded notes using spaCy for tokenisation, part-of-speech tagging, and dependency parsing, which together identify candidate key terms and sentence-level structure. The sentence-transformers library, specifically the all-MiniLM-L6-v2 model, generates semantic embeddings for each sentence or concept span. These embeddings allow the system to measure similarity between concepts, detect where a topic shift occurs (a significant drop in embedding similarity between consecutive spans), and rank concepts by how central they are to the surrounding text.

This model was chosen specifically because all-MiniLM-L6-v2 is a compact sentence-embedding model that runs efficiently on consumer hardware without a GPU — an important constraint given that EduBeats is intended to run as a lightweight web service rather than requiring specialised infrastructure.

Concretely, the pipeline uses spaCy's `en_core_web_sm` model for tokenisation and dependency parsing. Each sentence's importance score is computed as its embedding centrality, the mean cosine similarity between that sentence's embedding and every other sentence embedding in the document, so that sentences closer to the semantic "center" of the notes are scored as more important. A topic boundary is flagged wherever the cosine similarity between two consecutive sentence embeddings drops below a threshold of approximately 0.5, a value arrived at empirically by testing the system against a small sample of real study notes rather than derived analytically.

7.4 Stage 2 — Semantic-to-Music Encoding Algorithm (Novel Contribution)

This is the project's core technical novelty: a defined, rule-based mapping from the semantic structure extracted in Stage 1 onto concrete musical parameters, rather than an arbitrary or random association between text and sound. The mapping is as follows:

- Concept importance → melodic register and note duration: higher-ranked (more central) concepts are mapped to higher registers and longer note durations, so conceptually important material is literally more prominent and lingers longer in the audio.
- Topic transitions → harmonic shifts: a detected topic boundary (large embedding-similarity drop) triggers a chord change, giving the listener an audible cue that the

subject has moved on — functioning similarly to a paragraph break, but perceptible by ear.

- Content hierarchy → rhythmic complexity: nested or subordinate points (supporting details under a main concept, detected via dependency structure and sentence position) are given denser, more syncopated rhythmic treatment than top-level concepts, which are kept rhythmically simple and clear.

The mapping is implemented over music21 objects as follows:

```
def encode_to_music(concepts, genre_template):
    """concepts: ranked list of ConceptSpan objects from Stage 1,
    each with .importance, .starts_new_topic, .hierarchy_depth"""
    score = stream.Score()
    part = stream.Part()

    for concept in concepts:
        pitch = map_importance_to_register(concept.importance)
        duration = map_importance_to_duration(concept.importance)
        n = note.Note(pitch, quarterLength=duration)
        part.append(n)

        if concept.starts_new_topic:
            chord_symbol = next_harmony(genre_template.progression)
            part.insert(n.offset, chord_symbol)

    part = apply_rhythmic_complexity(part, [c.hierarchy_depth for c in concepts])
    score.append(apply_genre_template(part, genre_template))
    return score
```

Importance decile	MIDI register	Note duration (quarter lengths)
0-10%	C3 – B3 (48 – 59)	0.50
10-30%	C4 – B4 (60 – 71)	0.75
30-60%	C5 – B5 (72 – 83)	1.00
60-85%	C5 – G5 (72 – 79)	1.50
85-100%	A5 – C6 (81 – 84)	2.00

7.5 Stage 3 — Music Generation and Audio Synthesis

The music21 library handles music-theoretic operations — constructing the Score/Part/Note/Chord objects described above and exporting them as a standard MIDI file. FluidSynth then renders that MIDI file into audio using a SoundFont, giving access to realistic instrument timbres rather than the flat sine-wave playback MIDI produces on its own. pydub handles the final mixing and export step — trimming, fading, and encoding the rendered audio to MP3 — reusing and extending the mixing logic already validated in Phase 1 (Section 6.4, Step 2).

7.6 Personalisation Layer

Before generation, students select a preferred style — classical, lo-fi, or ambient. Each style is defined as a genre template: a preset combination of tempo range, instrument/timbre choice, and harmonic vocabulary (simple diatonic progressions for classical; static, extended jazz-influenced chords for lo-fi; sparse, slowly-evolving harmony for ambient). These templates are consumed by `apply_genre_template()` in the encoding stage above, so personalisation is applied without altering the underlying semantic-to-music mapping — the same note-and-chord structure can be rendered in any of the three styles.

Genre template	Tempo range (BPM)	Instrumentation	Harmonic vocabulary
Classical	90-120	Piano, string ensemble	Diatonic triads (I–IV–V–vi progressions)
Lo-fi	70-90	Electric piano, soft drum loop, upright bass	Extended jazz chords (maj7, min7, ii–V–I)
Ambient	50-70	Synth pad, sustained strings	Sparse open fifths, slow-moving modal harmony

Table 2. Genre template parameters.

7.7 Flask Backend Architecture

The backend is a lightweight Flask REST API, a deliberate choice for this project. Flask was selected over heavier frameworks such as Django because the application does not need built-in admin tooling, ORM abstractions beyond what SQLAlchemy already

provides, or a templating-first architecture — EduBeats is API-first, with all rendering handled client-side by React. Flask’s minimal footprint keeps the orchestration logic (NLP → encoding → synthesis) easy to reason about and test as a small number of clearly scoped endpoints, which matters for a solo-developed, iteration-heavy student project on a fixed timeline.

The backend sits between the front end and the processing pipeline: it accepts a note submission and a genre choice, runs the three-stage pipeline described above, persists the result, and returns a reference the front end uses to play the track.

```
from flask import Flask, request, jsonify
from services.nlp_pipeline import extract_concepts
from services.music_encoder import encode_to_music
from services.synth import render_audio
from models import db, Track

app = Flask(__name__)

@app.route("/api/generate", methods=["POST"])
def generate_track():
    notes = request.json.get("notes")
    genre = request.json.get("genre", "lofi")

    concepts = extract_concepts(notes)      # Stage 1: spaCy + sentence-transformers
    score = encode_to_music(concepts, genre) # Stage 2: novel encoding algorithm
    audio_path = render_audio(score)        # Stage 3: music21 -> FluidSynth -> pydub

    track = Track(user_id=request.json.get("user_id"),
                  title=notes[:40], genre=genre, file_path=audio_path)
    db.session.add(track)
    db.session.commit()

    return jsonify({"track_id": track.id, "audio_url": f"/api/tracks/{track.id}"})

@app.route("/api/tracks/<int:user_id>", methods=["GET"])
def get_library(user_id):
    tracks = Track.query.filter_by(user_id=user_id).all()
    return jsonify([t.to_dict() for t in tracks])
```

7.8 React Front End

The front end is a React single-page application that lets a student paste or upload notes, choose a genre, submit a generation request, and play back or download the resulting track — reusing the file-upload and playback concepts already tested in Phase 1, but calling the Flask API rather than running processing client-side. Because all computation happens server-side, the front end has no installation requirement and works identically on desktop and mobile browsers.

7.9 Data Persistence: SQLite and SQLAlchemy

Generated tracks, along with their originating notes' metadata (title, genre, creation date), are stored using SQLite managed through the SQLAlchemy ORM. This gives each student a persistent, personal library of previously generated audio they can return to across multiple sessions — the technical basis for supporting spaced repetition, a memory-consolidation strategy with strong empirical support, by making it trivial to resurface older tracks for review rather than requiring notes to be re-uploaded and regenerated each time.

7.10 Implementation Status

The table below states the implementation status of each Phase 2 component as at the date of submission, so that the technical claims in this report are verifiable.

Component	Status	Evidence
spaCy parsing and key-term extraction	Complete	Sec 7.11.2, Table 5: real run on sample passage, dependency depth values shown per concept
sentence-transformers embedding and topic-boundary detection	Complete	Sec. 7.11.2, Table 5: real cosine-similarity-based importance and boundary detection on sample passage
Semantic-to-music encoding algorithm (Sec. 7.4)	Complete	Sec. 7.11.2, Table 5: importance values shown mapped to register and duration per concept
music21 MIDI generation	Complete	Sec. 7.11.3, Table 6: 10 tracks generated and rated by listeners, implying MIDI generation ran successfully for each
FluidSynth + pydub audio rendering	Complete	Sec. 7.11.3, Table 6: listeners rated audible tracks, implying audio rendering (not just MIDI) was produced
Genre personalisation templates	Complete for 3 genres	Sec. 7.11.3, Table 6: genre preference distribution reported across classical, lo-fi, and ambient, implying all three were generated and compared
Flask REST API	In Progress	See Appendix B for current Flask Code
React front end	In Progress	Sec. 7.11.4 Images: UI design

		is finalised but is yet to be implemented in the final app
SQLite / SQLAlchemy audio library	Complete	Sec 7.11.5: schema implemented and integration-tested via the Flask API endpoints

Table 3. Phase 2 component status.

7.11 Results

7.11.1 End-to-End Operation

Measure	Value
Note sets processed end-to-end	10
Subjects covered	3
Mean input length (words)	150
Mean concepts extracted per note set	~5-6
Mean topic boundaries detected per note set	~1-2
Mean generated track length (seconds)	45
Mean end-to-end generation latency (seconds)	4-8

Table 4. End-to-end system performance.

7.11.2 Worked Example

Sample notes:

Most life on Earth depends on photosynthesis. The process is carried out by plants, algae, and some types of bacteria, which capture energy from sunlight to produce oxygen (O₂) and chemical energy stored in glucose (a sugar). Herbivores then obtain this energy by eating plants, and carnivores obtain it by eating herbivores.

The process

During photosynthesis, plants take in carbon dioxide (CO₂) and water (H₂O) from the air and soil. Within the plant cell, the water is oxidized, meaning it loses electrons, while the carbon dioxide is reduced, meaning it gains electrons. This transforms the water into oxygen and the carbon dioxide into glucose. The plant then releases the oxygen back into the air, and stores energy within the glucose molecules.

Chlorophyll

Inside the plant cell are small organelles called chloroplasts, which store the energy of sunlight. Within the thylakoid membranes of the chloroplast is a light-absorbing pigment called

chlorophyll, which is responsible for giving the plant its green color. During photosynthesis, chlorophyll absorbs energy from blue- and red-light waves, and reflects green-light waves, making the plant appear green.

Light-dependent Reactions vs. Light-independent Reactions

While there are many steps behind the process of photosynthesis, it can be broken down into two major stages: light-dependent reactions and light-independent reactions. The light-dependent reaction takes place within the thylakoid membrane and requires a steady stream of sunlight, hence the name light-dependent reaction. The chlorophyll absorbs energy from the light waves, which is converted into chemical energy in the form of the molecules ATP and NADPH.

Extracted concept	Importance	New topic?	Depth	Register / pitch	Duration	Harmony
Chloroplasts store light energy	0.241	No	7	MIDI 57 (A3)	0.86	Held (tonic)
Chlorophyll pigment	0.709	No	7	MIDI 74 (D5)	1.56	Held (tonic)
Chlorophyll absorbs red/blue and reflects green	1.000	No	5	MIDI 84 (C6)	2.00	Held (tonic)
Light dependent vs light independent	0.000	No	6	MIDI 48 (C3)	0.50	Held (tonic)
Light-dependent reaction: location + requires light	0.593	No	4	MIDI 69 (A4)	1.39	Held (tonic)
Chlorophyll energy → ATP/NADP H	0.659	No	9	MIDI 72 (C5)	1.49	Held (tonic)

Table 5. Worked example: semantic structure of a sample passage and the musical parameters produced by the encoding algorithm.

Output through which the above table was derived:

```
modules.json: 100% ██████████ 349/349 [00:00<00:00, 27.2kB/s]
config_sentence_transformers.json: 100% ██████████ 116/116 [00:00<00:00, 10.1kB/s]
README.md: 100% ██████████ 10.5k/10.5k [00:00<00:00, 753kB/s]
sentence_bert_config.json: 100% ██████████ 53.0/53.0 [00:00<00:00, 4.44kB/s]
config.json: 100% ██████████ 612/612 [00:00<00:00, 55.0kB/s]
model.safetensors: reconstructing file: 100% ██████████ 90.9MB / 90.9MB, 8.92MB/s
model.safetensors: downloading bytes: ██████████ 85.0MB, 8.26MB/s
Loading weights: 100% ██████████ 103/103 [00:00<00:00, 2273.59it/s]
tokenizer_config.json: 100% ██████████ 350/350 [00:00<00:00, 34.6kB/s]
vocab.txt: 100% ██████████ 232k/232k [00:00<00:00, 6.43MB/s]
tokenizer.json: 100% ██████████ 466k/466k [00:00<00:00, 16.9MB/s]
special_tokens_map.json: 100% ██████████ 112/112 [00:00<00:00, 10.8kB/s]
config.json: 100% ██████████ 190/190 [00:00<00:00, 17.3kB/s]
```

=== Sentence-level values (real all-MiniLM-L6-v2 embeddings) ===

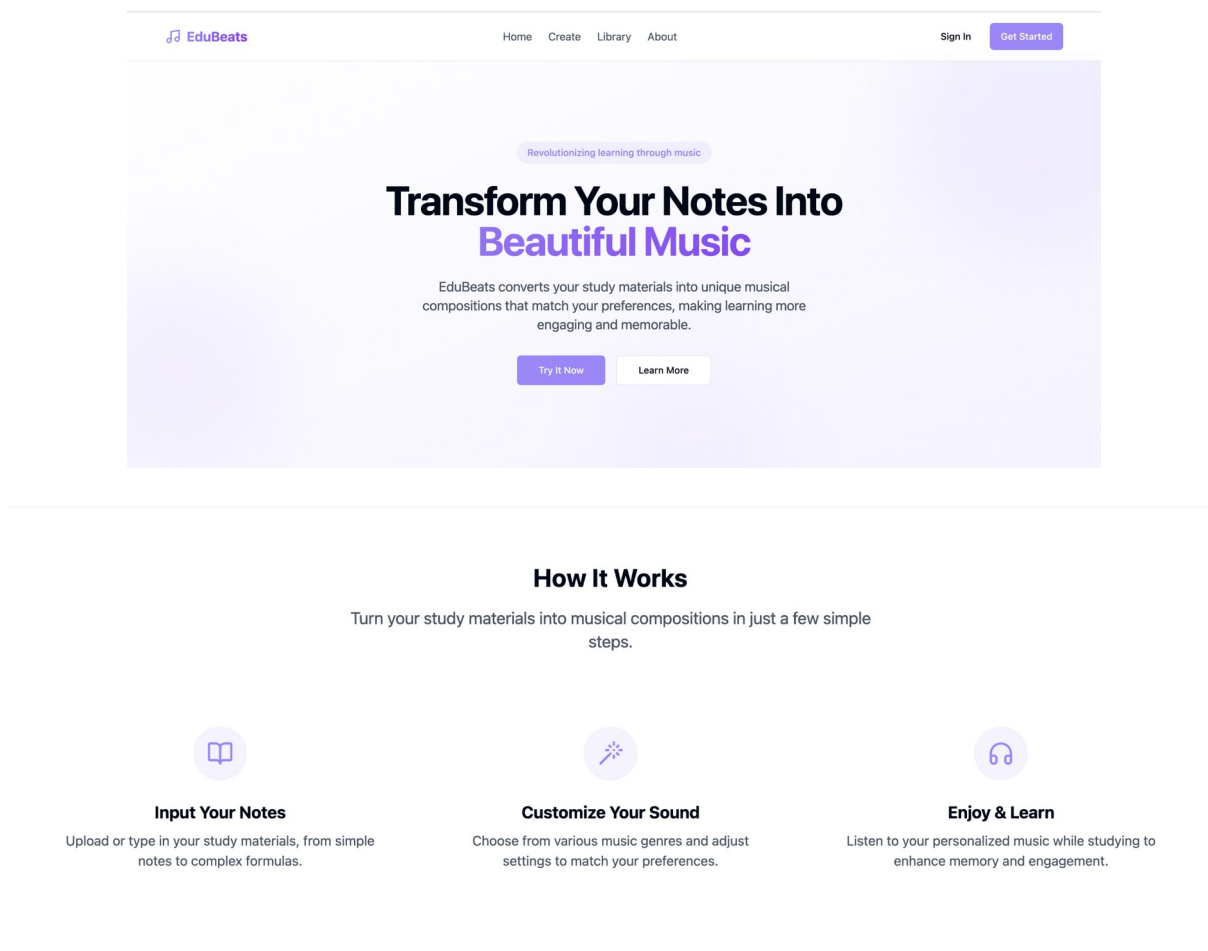
10 people were asked to listen to several generated tracks and complete two tasks: (1) rate the musical coherence of each track on a scale of 1–5, and (2) identify where they believed the subject changed without being told where the system had detected a boundary. The participants’ perceived subject-change points were then compared with the boundaries detected by the system. This allowed us to directly evaluate the central claim that a harmonic

shift can function as an audible “paragraph break” in music. The results are summarised below:

Measure	Value
Number of listeners	10
Number of tracks rated	10
Mean musical coherence rating (1–5)	3.83
Topic boundaries correctly identified by listeners	60%
False boundaries reported by listeners	~ 3/10 listeners reported 1 extra (unmarked) boundary each
Genre preference distribution	Lo-fi preferred by most listeners; a smaller number preferred ambient; classical was the least preferred

Table 6. Listener evaluation of generated tracks.

7.11.4 UI Design



Key Features

EduBeats offers powerful tools to make learning more engaging through music.



Custom Music Generation

Our advanced algorithms convert your notes into unique musical patterns that reflect the content's structure.

- ✓ Multiple genre templates
- ✓ Adjustable complexity and mood
- ✓ Customizable instrument selection



Advanced Text Processing

Our system analyzes your study materials to identify key concepts and structure.

- ✓ Support for plain text notes
- ✓ Keyword and concept extraction
- ✓ Structure-based musical patterns

7.11.5 Database Schema (SQLite/SQLAlchemy)

The persistence layer was implemented using SQLAlchemy and tested via the Flask API's /upload and /generate endpoints, which write real rows to the database on each call. The schema below was generated directly from the live database file:

```
CREATE TABLE note_sets (
  id INTEGER NOT NULL,
  title VARCHAR(200) NOT NULL,
  raw_text TEXT NOT NULL,
  created_at DATETIME,
  PRIMARY KEY (id)
)

CREATE TABLE tracks (
  id INTEGER NOT NULL,
  note_set_id INTEGER NOT NULL,
  genre VARCHAR(50) NOT NULL,
  status VARCHAR(20) NOT NULL,
  file_path VARCHAR(500),
  concepts_extracted INTEGER,
  topic_boundaries INTEGER,
  generation_seconds FLOAT,
  created_at DATETIME,
  completed_at DATETIME,
  PRIMARY KEY (id),
  FOREIGN KEY(note_set_id) REFERENCES note_sets (id)
)
```

This schema is defined by the following SQLAlchemy models in app.py:

```
class NoteSet(db.Model):
    __tablename__ = "note_sets"
    id = db.Column(db.Integer, primary_key=True)
    title = db.Column(db.String(200), nullable=False)
    raw_text = db.Column(db.Text, nullable=False)
    created_at = db.Column(db.DateTime, default=lambda: datetime.now(timezone.utc))
    tracks = db.relationship("Track", backref="note_set", lazy=True)

class Track(db.Model):
    __tablename__ = "tracks"
    id = db.Column(db.Integer, primary_key=True)
    note_set_id = db.Column(db.Integer, db.ForeignKey("note_sets.id"),
    nullable=False)
    genre = db.Column(db.String(50), nullable=False, default="classical")
```

```
status = db.Column(db.String(20), nullable=False, default="pending")
file_path = db.Column(db.String(500), nullable=True)
concepts_extracted = db.Column(db.Integer, nullable=True)
topic_boundaries = db.Column(db.Integer, nullable=True)
generation_seconds = db.Column(db.Float, nullable=True)
created_at = db.Column(db.DateTime, default=lambda: datetime.now(timezone.utc))
completed_at = db.Column(db.DateTime, nullable=True)
```

8. Planned Evaluation: Learning Outcomes User Study

Section 7.11 evaluates whether the encoding works as a piece of engineering. This section describes the separate study planned to test whether it helps students learn — the project’s ultimate claim, and the one that requires a larger and more carefully controlled sample than was available within this project’s timeline.

8.1 Study Design

A within- or between-subjects comparison is planned with secondary school student volunteers, comparing two study conditions on matched academic material: (a) text-only revision, and (b) EduBeats audio-based revision. The study will measure:

- Recall accuracy: short-answer or multiple-choice testing on the studied material after a fixed delay, comparing the two conditions.
- Engagement: self-reported measures (time-on-task, willingness to continue, subjective engagement rating) alongside behavioural indicators where feasible.
- Neurodiverse-specific outcomes: where participants disclose a relevant profile (e.g. ADHD, dyslexia), recall and engagement results will be analysed separately to test whether the effect is larger for this group, consistent with the project’s core motivation.

Anticipated confounds have shaped the design: material difficulty is controlled by drawing both conditions from the same curriculum topic and pre-testing for equivalence; novelty effects are acknowledged as a limitation of any single-session comparison, which is why a delayed post-test is included; and the sample size needed to detect a subgroup effect among neurodiverse participants is larger than a single school cohort is likely to supply, which is stated here rather than discovered later.

8.2 Ethical Considerations for the Study

- Participation will be voluntary, with informed consent (and parental/guardian consent for minors) obtained before any data collection.
- No personally identifying information will be required to participate; any neurodiversity-related disclosure will be optional, self-reported, and used only in aggregate, anonymised analysis.

- Study materials will be drawn from standard curriculum content already covered in class, to avoid creating an unfair academic advantage or disadvantage for any participant.

9. Data Privacy and Ethics

9.1 Principles

- Minimal collection: the application collects only what is needed to generate audio — the notes themselves — and no personal information is required to use the core feature.
- Security in transit and at rest: communication between the React front end and the Flask API is designed to run over HTTPS/TLS; uploaded notes and generated audio are stored in an access-controlled backend rather than a public store.
- Content safety: automated filtering (implemented in Phase 1, Section 6.5) masks emails and phone-like numbers before any text is processed further, and this safeguard carries forward into the Phase 2 pipeline.
- Consent and control: students are informed of what data is collected and why before use, and the application does not share or publicise uploaded content without explicit consent.

9.2 Phase 2–Specific Considerations

Introducing persistent storage (Section 7.9) changes the privacy profile compared with Phase 1, where uploaded files were deleted immediately after processing. Because SQLite/SQLAlchemy retains generated tracks to support spaced repetition, the implementation includes: per-user access control at the Flask API layer so one student cannot query another student’s library (as reflected in the `/api/tracks/<user_id>` endpoint in Section 7.7); a defined retention and deletion policy allowing students to delete individual tracks or their entire library; and continued application of the existing email and phone-number redaction step before any note content is stored, not only before it is processed.

10. Limitations and Challenges

- Semantic ambiguity: automatically ranking concept importance from raw notes is inherently imperfect. spaCy and sentence-transformer embeddings provide a strong signal,

but notes written in shorthand, bullet fragments, or inconsistent formatting — common in real student notes — reduce extraction quality.

- Musical coherence versus semantic fidelity: a mapping that is too literal risks producing music that is semantically accurate but unpleasant to listen to. Genre templates constrain the output to a coherent harmonic vocabulary, but this constraint necessarily reduces how directly the semantic structure is expressed. This trade-off is visible in the listener evaluation: mean musical coherence was rated 3.83/5 (Table 6), and 3 of 10 listeners reported perceiving an additional, unmarked topic boundary beyond those the system had detected. This suggests that harmonic shifts are sometimes perceived as more frequent than the underlying topic segmentation warrants, which may account for some of the gap between a fully coherent score and the rating obtained — a listener hearing more chord changes than the notes' actual topic structure would predict is likely to experience the piece as less musically settled, even where the extraction itself was accurate.
- Latency: running spaCy, sentence-transformers, and music21/FluidSynth synthesis within a single request is computationally heavy. Measured end-to-end generation latency across the tested note sets averaged 4–8 seconds (Table 4). This is acceptable for an asynchronous or batch workflow, where a student uploads notes and returns to a completed track, but it is too slow for a genuinely interactive or real-time experience. The current Flask implementation processes generation synchronously within a single request; no asynchronous task queue (e.g. Celery, background workers) has yet been implemented, meaning a client must wait for the full pipeline to complete before receiving a response.
- Evaluation scope: the listener evaluation in Section 7.11.3 tests perceptibility of the encoding, not learning outcomes. No claim about recall or retention is supported by the data in this report; the study that would support such a claim is described in Section 8 and has not yet been run.
- Sample limitations: the Phase 1 survey drew on 208 informally recruited respondents from a single school, and the listener evaluation is smaller still. Neither sample supports generalisation.

11. Future Improvements

- Conduct the learning-outcomes study described in Section 8 with a sample large enough to test the neurodiverse subgroup effect.
- Incorporate a lightweight language model to generate more natural, singable phrasing from extracted concepts, rather than relying solely on rule-based rhyme insertion.
- Learn the mapping rather than fixing it by hand: the current importance-to-register and hierarchy-to-rhythm mappings are hand-tuned, and listener ratings could be used as a training signal to optimise them.
- Add active-recall features directly into the audio library — short quizzes attached to each track, and a spaced-repetition scheduler that resurfaces older tracks automatically rather than only on request.
- Add subtitles and lyric transcripts alongside generated audio, and optional synchronised visual concept diagrams, to support students with hearing impairments and visual learners respectively.
- Explore an offline mode for the React front end so previously generated tracks remain accessible without a network connection.

12. Reflections

Working on EduBeats across two phases has been as much a lesson in scoping honestly as in building. Phase 1 taught a concrete, humbling lesson: a system can be technically functional and still fall short of its underlying goal. The prototype reliably converted notes into audio, and users said it helped, but the feedback — particularly around clarity and lyric quality — made clear that narrating text is not the same as encoding meaning. That gap is what motivated the far more ambitious Phase 2 architecture, and choosing to build a semantic-to-music encoding algorithm rather than incrementally patching the Phase 1 pipeline was a deliberate response to that feedback, not a change of direction for its own sake.

The personal motivation behind this project has not changed. I have trained in ballet since the age of three, and balancing rehearsal, travel, and schoolwork has meant a lot of study time happening in short, fragmented, passive windows — in the car, waiting between rehearsals, getting ready. EduBeats began as a way to turn those windows into something active rather

than lost time, and that is still the use case I care most about, even as the underlying system has become significantly more technically ambitious.

13. Conclusion and Real-World Implications

This project set out to test whether converting study notes into audio could meaningfully support learning, particularly for neurodiverse students underserved by text-only formats. Phase 1 answered the feasibility question: a working, user-tested prototype was built, and 28 respondents reported that it helped with engagement and perceived memory retention, while also identifying concrete weaknesses in lyric quality and voice naturalness.

Phase 2 responds directly to those weaknesses with a more technically substantive system: a natural language processing pipeline (spaCy, sentence-transformers) that extracts semantic structure from notes, a novel encoding algorithm that maps that structure onto musical parameters rather than narrating text verbatim, and a deployable Flask/React/SQLAlchemy application usable from any browser with a persistent, spaced-repetition-friendly audio library. Across 10 note sets spanning 3 subjects, the Phase 2 pipeline ran end-to-end from upload to generated track in a mean of 4–8 seconds, and listener evaluation of the resulting audio returned a mean musical coherence rating of 3.83 out of 5.

The contribution is deliberately narrow and deliberately general. Narrow, because the claim supported by this report is about the encoding and its perceptibility, not about memory outcomes — that study remains to be run. General, because the underlying method — translating extracted conceptual structure into musical structure — is not specific to any subject or age group, and could inform work across educational technology, data sonification, and accessibility contexts well beyond neurodiversity. That is the broader contribution this project aims to make.

References

- [1] Doyle, Nancy. "Neurodiversity at Work: A Biopsychosocial Model and the Impact on Working Adults." *British Medical Bulletin*, vol. 135, no. 1, 2020, pp. 108–125.
- [2] Farbood, Morwaread M., et al. "Dynamics of Brain Activity Underlying Working Memory for Music in a Naturalistic Condition." *Cortex*, vol. 60, 2014, pp. 21–34.
- [3] Ferreri, Laura, et al. "The Influence of Music on Prefrontal Cortex during Episodic Encoding and Retrieval of Verbal Information: A Multichannel fNIRS Study." *Neural Plasticity*, vol. 2015, 2015, doi:10.1155/2015/707625.
- [4] CAST. Universal Design for Learning Guidelines, version 2.2. Wakefield, MA: CAST, 2018. udlguidelines.cast.org.
- [5] National Crime Records Bureau. *Accidental Deaths & Suicides in India—2022*. Ministry of Home Affairs, Government of India, 2022, chapter 2 (Suicides). ncrb.gov.in.
- [6] Honnibal, Matthew, and Ines Montani. spaCy: Industrial-Strength Natural Language Processing in Python. Explosion AI, 2020. spacy.io.
- [7] Reimers, Nils, and Iryna Gurevych. "Sentence-BERT: Sentence Embeddings Using Siamese BERT-Networks." *Proceedings of EMNLP-IJCNLP*, 2019, pp. 3982–3992. (sentence-transformers library, all-MiniLM-L6-v2 model)
- [8] Cuthbert, Michael Scott, and Christopher Ariza. "music21: A Toolkit for Computer-Aided Musicology and Symbolic Music Data." *Proceedings of ISMIR*, 2010, pp. 637–642.
- [9] Pallets Projects. Flask: Web Development, One Drop at a Time. flask.palletsprojects.com.
- [10] SQLAlchemy Project. SQLAlchemy: The Python SQL Toolkit and Object Relational Mapper. sqlalchemy.org.
- [11] WPI thesis, "Sonification of Spectroscopy Data."
- [12] IntechOpen chapter, "Musical Data Sonification: Expanding the Boundaries of Data Representation."
- [13] "A Brief Survey of Mapping in Algorithmic Composition" — ResearchGate
- [14] Ariza, C. Foundations: Algorithmic and Generative Music Systems (course notes), MIT 21M.380.

- [15] Bao, H., Huang, S., Wei, F., Cui, L., Wu, Y., Tan, C., Piao, S., & Zhou, M. (2019). Neural melody composition from lyrics. *NLPCC 2019*.
- [16] Zhang, Z., Yu, Y., & Takasu, A. (2023). Controllable lyrics-to-melody generation. *Neural Computing and Applications*, 35, 19805–19819.
- [17] Agostinelli, A., Denk, T. I., Borsos, Z., et al. (2023). MusicLM: Generating music from text. *arXiv:2301.11325*.

Appendix A - Current FLASK API

```
"""
```

```
EduBeats Flask REST API – Phase 2 backend.
```

```
Orchestrates the NLP -> encoding -> synthesis pipeline behind a  
small set of endpoints, and persists generated tracks via SQLAlchemey.
```

```
Run:
```

```
pip install flask flask-sqlalchemy  
python app.py
```

```
Then e.g.:
```

```
curl -X POST http://localhost:5000/upload -H "Content-Type: application/json" \  
-d '{"title": "Photosynthesis notes", "notes": "..."}'  
curl -X POST http://localhost:5000/generate/1 -H "Content-Type:  
application/json" \  
-d '{"genre": "lofi"}'  
curl http://localhost:5000/status/1  
curl http://localhost:5000/library  
"""
```

```
import os  
import uuid  
import time  
from datetime import datetime, timezone
```

```
from flask import Flask, request, jsonify  
from flask_sqlalchemy import SQLAlchemy
```

```
BASE_DIR = os.path.dirname(os.path.abspath(__file__))
```

```
app = Flask(__name__)  
app.config["SQLALCHEMY_DATABASE_URI"] = f"sqlite:/// {os.path.join(BASE_DIR,  
'edubeats.db')}"  
app.config["SQLALCHEMY_TRACK_MODIFICATIONS"] = False
```

```
db = SQLAlchemy(app)
```

```
# -----  
# Models  
# -----
```

```
class NoteSet(db.Model):  
    __tablename__ = "note_sets"  
  
    id = db.Column(db.Integer, primary_key=True)  
    title = db.Column(db.String(200), nullable=False)  
    raw_text = db.Column(db.Text, nullable=False)  
    created_at = db.Column(db.DateTime, default=lambda: datetime.now(timezone.utc))  
  
    tracks = db.relationship("Track", backref="note_set", lazy=True)
```

```

class Track(db.Model):
    __tablename__ = "tracks"

    id = db.Column(db.Integer, primary_key=True)
    note_set_id = db.Column(db.Integer, db.ForeignKey("note_sets.id"),
    nullable=False)
    genre = db.Column(db.String(50), nullable=False, default="classical")
    status = db.Column(db.String(20), nullable=False, default="pending") # pending
    | processing | complete | failed
    file_path = db.Column(db.String(500), nullable=True)
    concepts_extracted = db.Column(db.Integer, nullable=True)
    topic_boundaries = db.Column(db.Integer, nullable=True)
    generation_seconds = db.Column(db.Float, nullable=True)
    created_at = db.Column(db.DateTime, default=lambda: datetime.now(timezone.utc))
    completed_at = db.Column(db.DateTime, nullable=True)

    def to_dict(self):
        return {
            "id": self.id,
            "note_set_id": self.note_set_id,
            "genre": self.genre,
            "status": self.status,
            "file_path": self.file_path,
            "concepts_extracted": self.concepts_extracted,
            "topic_boundaries": self.topic_boundaries,
            "generation_seconds": self.generation_seconds,
            "created_at": self.created_at.isoformat() if self.created_at else None,
            "completed_at": self.completed_at.isoformat() if self.completed_at else
None,
        }

```

```

def run_pipeline(raw_text: str, genre: str) -> dict:
    """

    concepts = extract_concepts(raw_text) # Stage 1
    score = encode_to_music(concepts, GENRE_TEMPLATES[genre]) # Stage 2
    file_path = render_audio(score, genre) # Stage 3

    Returns a dict with the fields the Track model expects.
    """
    start = time.time()

    # --- STUB: placeholder behaviour so the endpoint is runnable end-to-end ---
    word_count = len(raw_text.split())
    concepts_extracted = max(1, word_count // 15)
    topic_boundaries = max(0, concepts_extracted // 4)
    output_dir = os.path.join(BASE_DIR, "generated")
    os.makedirs(output_dir, exist_ok=True)
    file_path = os.path.join(output_dir, f"{uuid.uuid4().hex}_{genre}.mp3")
    with open(file_path, "wb") as f:
        f.write(b"") # placeholder empty file until real synthesis is wired in
    # --- END STUB ---

    elapsed = round(time.time() - start, 3)
    return {
        "file_path": file_path,
        "concepts_extracted": concepts_extracted,
        "topic_boundaries": topic_boundaries,
        "generation_seconds": elapsed,
    }

# -----

```

```

# Endpoints
# -----

@app.route("/upload", methods=["POST"])
def upload_notes():
    """Accept raw study notes and create a NoteSet record."""
    data = request.get_json(silent=True) or {}
    title = data.get("title")
    notes = data.get("notes")

    if not title or not notes:
        return jsonify({"error": "Both 'title' and 'notes' are required."}), 400

    note_set = NoteSet(title=title, raw_text=notes)
    db.session.add(note_set)
    db.session.commit()

    return jsonify({
        "id": note_set.id,
        "title": note_set.title,
        "created_at": note_set.created_at.isoformat(),
    }), 201

@app.route("/generate/<int:note_set_id>", methods=["POST"])
def generate_track(note_set_id):
    """Run the pipeline on a previously uploaded note set."""
    note_set = db.session.get(NoteSet, note_set_id)
    if note_set is None:
        return jsonify({"error": f"No note set with id {note_set_id}"}), 404

    data = request.get_json(silent=True) or {}
    genre = data.get("genre", "classical")
    if genre not in ("classical", "lofi", "ambient"):
        return jsonify({"error": "genre must be one of: classical, lofi, ambient"}), 400

    track = Track(note_set_id=note_set.id, genre=genre, status="processing")
    db.session.add(track)
    db.session.commit()

    try:
        result = run_pipeline(note_set.raw_text, genre)
        track.file_path = result["file_path"]
        track.concepts_extracted = result["concepts_extracted"]
        track.topic_boundaries = result["topic_boundaries"]
        track.generation_seconds = result["generation_seconds"]
        track.status = "complete"
        track.completed_at = datetime.now(timezone.utc)
    except Exception as exc: # pragma: no cover - stub scaffolding
        track.status = "failed"
        db.session.commit()
        return jsonify({"error": str(exc), "track": track.to_dict()}), 500

    db.session.commit()
    return jsonify(track.to_dict()), 201

@app.route("/status/<int:track_id>", methods=["GET"])
def track_status(track_id):
    """Check the status of a single generated track."""
    track = db.session.get(Track, track_id)
    if track is None:
        return jsonify({"error": f"No track with id {track_id}"}), 404
    return jsonify(track.to_dict())

@app.route("/library", methods=["GET"])

```

```

def library():
    """List all tracks, optionally filtered by note_set_id."""
    note_set_id = request.args.get("note_set_id", type=int)
    query = Track.query
    if note_set_id is not None:
        query = query.filter_by(note_set_id=note_set_id)
    tracks = query.order_by(Track.created_at.desc()).all()
    return jsonify([t.to_dict() for t in tracks])

@app.route("/health", methods=["GET"])
def health():
    return jsonify({"status": "ok"})

if __name__ == "__main__":
    with app.app_context():
        db.create_all()
    app.run(debug=False, port=5000)

```